Viva notes

Short Solutions for Large Rubik's Cubes and Squares

Prepared from the paper and the code at `github.com/gauravrai1704/Rubik`

14 November 2026

Contents

1 The paper in one page 2

2 Problem, puzzles, metrics and data 2

3 The engine (`rubiks/engine`) 3

4 The methods 4

5 SymPort in depth (the main method) 8

6 The counting lower bound 10

7 Experiments and statistics 10

8 Code map and how to run things 11

9 Question bank 11

10 Weak spots and inconsistencies to be ready for 13

11 Future work (as in the paper) 13

1 The paper in one page

One sentence. We build one open engine for the $N\times N\times N$ Rubik's Cube and the $N\times N\times 1$ Rubik's Square, run eight solvers (five reproductions, three new) and three external baselines on the same instances, and find that (i) our symmetry portfolio SymPort beats the best published competition solutions on average for cubes $N = 4$–7, (ii) our NetGain schedule is the best single solver on large random states, and (iii) everything is still a factor ≈ 2 above a counting lower bound, a gap that grows like $\log N$.

The four research questions (intro): (1) what does it take to cover every size, for both puzzles; (2) which approach is shortest at each size; (3) can any method beat the published competition solutions; (4) how far is everything from optimal.

Numbers to know by heart.

Santa cube puzzles used (standard goal, no wildcards, $N \le 10$)	184 (N=2:20, 3:82, 4:43, 5:19, 6:6, 7:4, 8:3, 9:3, 10:4)
Table 2 (Santa) sizes	$N = 4$–9 (78 puzzles); we stopped after two losses
SymPort vs KMC-published, mean QTM	4: 64.1/66.7; 5: 95.0/97.1; 6: 158.3/161.0; 7: 205.2/208.8
	8: 294.7/**291.3**; 9: 361.3/**354.7**
Totals, $N = 4$–7 (72 puzzles)	6,334 vs 6,514 QTM (−2.8%), W/T/L 41/14/17
Totals, $N = 4$–9 (78 puzzles)	8,302 vs 8,452 QTM (−1.8%), W/T/L 43/14/21
KMC-one run (one run of their program)	77.0, 114.2, 179.3, 232.8, 310.7, 390.0 for $N = 4..9$
SymPort on Squares	best at all 18 sizes $N = 3$–20
NetGain vs Demaine, random cubes	20 states: −17.2% at N=10; 2-state main table: 21% at 10, 26% at 20
God's numbers (Square, colour goal)	6, 10, 12 for $N = 3, 4, 5$ (bound 4, 6, 8; ratio 1.5–1.7)
Published Santa / counting bound	1.81× (N=4), 1.99× (7), 2.22× (10)
SymPort cost per copy	0.3 CPU-min (N=4) → 31 (8) → 47 (9)
Hardware	4-core, 15 GB CPU machine; Colab T4 GPU for the GPU-trained models

The names (the paper uses names, not numbers; the code uses numbers):

Paper name	Code	What	Ours?
IDA*-PDB	`m1_idastar`	Korf's IDA* + pattern databases; BFS God's numbers	reproduction
Reduction	`m2_reduction`	centers, 3×3×3 by Kociemba two-phase, wings	reproduction
Demaine	`m3_demaine`	Demaine et al. parallel cluster algorithm (3-cycle batching on cube)	reproduction
CayleyPy	`m4_cayleypy`	per-size ResMLP distance + beam search	reproduction
OneNet	`m5a_tokens`	one cluster-token transformer for all sizes	new
NetGain	`m5c_schedule`	net-gain scheduling of bulk commutators	new
KMC-Repro	`m6_kmcoders`	KMCoders pipeline rebuilt from the write-up	reproduction
SymPort	`m7_portfolio`	symmetry portfolio (main method)	new
KMC-published	`santa_kmcoders`	KMCoders' final Santa submission	baseline
KMC-one run	`external/kmcoders_run`	one run of KMCoders' Rust program	baseline
dwalton76	`external/dwalton76`	open NxNxN reduction solver	baseline

`m5b_stages` in the code is an earlier staged-cluster method ("OneNet-B") that you removed from the final paper; it is still in the repository and its results files. Agree on a one-line reason before the viva in case an examiner opens that folder.

2 Problem, puzzles, metrics and data

2.1 The two puzzles

- **Cube** N×N×N: $6N^2$ stickers, $3N$ slices (3 axes × N layers). Each slice can turn a quarter, half or three-quarter turn: $9N$ moves in HTM.
- **Square** N×N×1 (Demaine et al. 2011): an $N \times N$ grid, each cubie with a top and a bottom colour, plus border side stickers. $2N$ moves, each a 180° flip of a row or column; each is its own inverse; row flips commute with row flips, column flips with column flips. Same cluster structure as the cube, but small enough to compute exact answers for $N \le 5$. That is why we use it: it is the calibration ground for optimality and for learned methods.

2.2 Clusters (orbits)

A cubie can only reach positions in its own cluster; moves never mix clusters. Solving the puzzle = solving every cluster.

- Cube: 1 corner cluster (24 stickers); 1 middle-edge cluster if N odd; $k = \lfloor N/2 \rfloor - 1$ wing clusters (48 stickers each); $((N-2)^2 - [N \text{ odd}])/4$ center clusters (24 stickers: $+$, $\times$, oblique); 1 fixed-center cluster if N odd (6). A 20×20×20: 2,400 stickers, 9 wing and 81 center clusters.
- Square: 1 corner (16), $2k$ edge (12), k^2 interior (8); edge-cross, cross, center (6, 4, 2) for odd N only.

2.3 Metrics

- **HTM** (half-turn metric): any turn of one slice = 1. Used for random states.
- **QTM** (quarter-turn metric): a half turn = 2. **The Santa 2023 score is exactly QTM**, since every allowed Santa move is a quarter turn of one slice.
- **STM** (slice-turn metric): a block of adjacent layers turned together = 1. Reported, not used for ranking.
- On the Square all moves are flips, so HTM = QTM.
- Lengths are measured *after canonicalisation* (merging commuting same-axis moves).
- Cross-size comparison: divide by $N^2/\ln N$ (the diameter's growth rate) or by the counting bound.

2.4 Goals

Random benchmark: solved = every face one colour, *any* reachable orientation (24 for the cube, 4 for the Square). Santa: *one exact state* (fixed orientation). That is why some methods need extra "orientation" moves for Santa.

2.5 Data

Random states. End of a long random canonical walk from solved ($\geq 25\times$ the number of moves and ≥ 400 moves); approximates a uniform random state (not an exact uniform sampler — a stated limitation). Seeded, regenerated, and every state passes the solvability checker. Main tables: 10 Squares per size for $N \leq 12$, 5 for 13–20; 2 cubes per size for $N \leq 10$, one each at 15 and 20. Statistics section: 20 states per size, cubes $N = 3$–10 and every Square size.

Santa 2023 (Kaggle, Dec 2023–Jan 2024): 398 puzzles (cubes, wreaths, globes), each a start state, goal state and allowed moves as sticker permutations; score = total moves. We keep the 184 cube puzzles $N \leq 10$ with one colour per face and no wildcards. We left out 19×19×19 and 33×33×33 (one SymPort copy would take hours). The adapter derives the Santa-to-engine sticker map from the move permutations themselves (each Santa move must equal exactly one engine move) and is checked by replaying KMCoders' published solutions, which all reach the exact goal.

3 The engine (`rubiks/engine`)

Everything is generic over cuboid dimensions (d_0, d_1, d_2): the cube is (N, N, N), the Square $(N, N, 1)$, same code.

Moves from geometry (`puzzle.py`, `moves.py`)
Each sticker stores a doubled, centred cubie coordinate and an outward normal. A move rotates every sticker of one slab by an integer rotation matrix; looking up the rotated (position, normal) gives a permutation π; applying a move is the gather $s \leftarrow s[\pi]$, $O(6N^2)$. Non-square cross-sections (a row of the Square) only get 180° turns. Tests: every move has order 4 (or 2); move then inverse = identity.

Canonical form
Same-axis moves commute. Merge same-axis runs per layer and sort by layer. Also used to prune search: after a move on axis a, layer ℓ, no move on axis a with layer $\leq \ell$. Removes repeats and redundant orderings; it is exactly the word model of the counting bound.

Clusters with N-invariant signatures (`orbits.py`)
Union-find over all move permutations → sticker orbits; orbits sharing cubies merged into cubie clusters. Stickers inside a cluster are sorted by face and by signs and relative magnitudes of in-face coordinates — an order that does not depend on N. So the moves acting on a cluster induce the same local permutations at every N; we hash generators + solved pattern into a *signature*. Tests: every ×-center of 5×5×5, 7×7×7, 9×9×9 has the same signature. *This is what lets one network (*OneNet*) work at every size.*

Projections (`projection.py`)
Restrict to a subset of stickers; if small, BFS it: exact distances for that subset and an exact membership test.

Solvability checker (`checker.py`)
Necessary conditions: colour counts; projection membership for every small cluster (all Square clusters); valid corner/edge/wing pieces; corner twist $\equiv 0 \pmod 3$; for odd N even middle-edge flip and equal corner/middle-edge permutation parity; Square cross-cluster consistency. Tests reject a single twist, flip, two-corner swap, wrong colour count. *Necessary, not sufficient* — a stated limitation.

Harness (`bench/harness.py`)
Every solver implements one interface (`solvers/base.py`) and returns engine moves. The harness re-applies them, decides "solved" itself, writes one JSONL row per instance (HTM, QTM, STM, wall time, nodes, memory, seed, hardware, git commit). Crash or timeout = failure, stays in the denominator. External programs (KMCoders, dwalton76, nxopt) run as subprocesses; wide turns expanded; re-verified.

Shared beam search (`search/beam.py`)
Used by CayleyPy and OneNet. Expand in chunks, score children, keep the W best after dedup by a random 64-bit linear hash (also across the last 3 steps). Store only (score, hash, parent, action); rebuild next beam from parents. States packed at 3 bits/sticker; can live on GPU, pinned host memory or disk. Supports macro actions. One step: $O(WA)$ heuristic evaluations.

Pure 3-cycles for any cluster (`cluster_tools.py`, `search/macros.py`)
Centers: commutators $[A, B\,C\,B']$ with A, C inner-slice quarter turns on one axis and B an outer face turn. Wings: $[X, Y\,Z\,Y']$, X inner slice, Y, Z outer faces. Keep a candidate if it moves exactly three cubies of one cluster. A conjugate $s\,c\,s'$ of a pure 3-cycle is pure, so BFS over setups (up to 4 moves) extends a few base commutators to all $24 \cdot 23 \cdot 22/3 = 4{,}048$ 3-cycles of a cluster.

Why 4,048? Ordered triples of distinct positions $24 \cdot 23 \cdot 22$; each 3-cycle is counted 3 times (rotations of the cycle). Both directions of a cycle are distinct 3-cycles.

4 The methods

4.1 IDA*-PDB (reproduction of Korf 1997)

Idea. Optimal search: iterative-deepening A* with an admissible heuristic from pattern databases.

- PDBs for 3×3×3 by BFS over cubie coordinates: 8 corners ($8! \cdot 3^7 = 88{,}179{,}840$ entries) and two sets of 6 edges ($12!/6! \cdot 2^6 = 42{,}577{,}920$ each). Max entries 11 (corners), 10 (edges), HTM. Under 200 MB at 4 bits/entry. Move tables derived from engine permutations, cross-checked by a test. Files: `checkpoints/m1/pdb_{htm,qtm}_*.npy`.
- h = max of the three lookups: admissible (each is a relaxation) and consistent.
- IDA*: thresholds $\theta = h(s_0), \ldots$; prune $g + h > \theta$ and by canonical order. Numba, parallel over root moves, $\approx 1.2 \cdot 10^7$ nodes/s on 4 cores.
- Square: h = max over clusters of the cluster's exact distance (projection tables); admissible because no solution fixes a cluster faster than that cluster alone needs. Runs to each of the 4 solved orientations, keeps the shortest.
- God's numbers: BFS from all four solved Squares, $N \le 5$ (5×5×1: $1.06 \cdot 10^7$ states, ≈6 GB in Python).

Results. Optimal on all random Squares $N \le 5$ (5×5×1 in about 13 s each); 0/10 at $N = 6$. 3×3×3: 91/100 twelve-move scrambles; no random 3×3×3 (depth ≈ 18) in 30 minutes. Cost $O(b^{d-\bar{h}})$, $b \approx 13.3$ canonical HTM branching.

Q: Why does your IDA* not solve random 3x3x3 when Korf did in 1997?

A: Korf used much more time per instance (days on his hardware) and his runs averaged many hours; we cap at 30 minutes on four cores. Our PDBs match his sizes exactly; it is a budget limit, not a correctness issue. 12-move scrambles are solved optimally.

4.2 Reduction (standard reduction + Kociemba two-phase)

Order: orient (relabel colours; for Santa, middle-slice turns bring fixed centers home) → wing parity (one inner-slice quarter turn per odd wing cluster, Demaine Lemma 7, because 3-cycles are even permutations) → centers by greedy pure 3-cycles (most pieces fixed per move) → 3×3×3 stage (corners, middle edges, fixed centers) by Kociemba's two-phase (`kociemba` package; even N: odd corner permutation fixed first by one outer quarter turn, "PLL parity") → wings by greedy 3-cycles. Stages never undo each other: outer turns keep centers on their faces; pure 3-cycles touch nothing else.

Square: frame and edge/cross clusters solved exactly on their projections (Demaine Lemma 3, $O(N)$ moves); each interior cluster with its commutators $c\,r\,c\,r$ (Lemma 2), at most three each.

Cost: $\Theta(N^2)$ moves (each 3-cycle fixes $O(1)$ pieces). 1,885 HTM on random 10×10×10 (20-state mean).

4.3 Demaine (reproduction of Demaine et al., ESA 2011)

Idea. The only method with the optimal asymptotic rate $\Theta(N^2/\log N)$: one shared move sequence fixes many clusters at once.

- **Square (faithful).** Read each interior cluster's optimal commutator word from its table; round t applies every cluster's t-th commutator; clusters needing the same one are covered by blocks of columns X and rows Y fixed by one bulk sequence $C_X R_Y C_X R_Y$ ($2|X| + 2|Y|$ moves, Lemma 4), with Demaine's column groups (Lemmas 5–6).
- **Cube (adapted).** Demaine's batching by identical cluster configuration is useless at practical sizes (identical states are too rare, $\approx 3 \cdot 10^{15}$ states per center cluster). We batch 3-cycles instead, as KMCoders suggest. A *type* = outer-face setup + template with layer parameters, e.g. $[A_i, B\,C_j\,B']$; with layer sets I, J (all slices of I turned together) it applies the same 3-cycle to every cluster $(i, j) \in I \times J$ for $2|I| + 2|J| + 4$ moves. Each round: the type with most pieces fixed per move, applied to every block of clusters it helps *and harms none*; or a single 3-cycle if better.

Weakness that motivates NetGain. Blocks require unanimous need. Clusters diverge, blocks shrink to single cells: on the 20×20×20, 244 of 292 center rounds fixed one cluster. Our cube batching has no $N^2/\log N$ guarantee.

4.4 CayleyPy (reproduction of Chervov et al., NeurIPS 2025)

Training (`m4_cayleypy/train.py`).

- Data: fresh *non-backtracking* random walks from solved (never the inverse of the last move); each visited state labelled with its walk step k (a "diffusion distance", an upper-bound-ish proxy for true distance). Fresh data every epoch, so no overfitting to a fixed set.
- Walk horizon $k_{\max}$: paper's 26/45/65 for cubes 3/4/5; Squares $\approx 2.5\,N^2/\ln N + 4$.
- Model: `ResMLP` on one-hot stickers: Linear(N_1)+BN+ReLU, Linear(N_2)+BN+ReLU, N_r residual blocks of two Linear(N_2)+BN, scalar output. Presets: paper3 (700, 643, 4) = 4M params; **small (256, 192, 2)** for our CPU Square models.
- Loss MSE on k; Adam, fixed lr 10^{-3}; batch 1024; epochs of 1M fresh examples.
- Square models: 0.25–0.45M parameters, 4M examples ($N \le 5$) or 10M (N = 6–8), 1–4 CPU-minutes each. A "large" GPU set (1.27M params, 256M examples at $N = 8$) on a Colab T4; the GPU model is the default at $N = 8$.

Search. Beam search, width W (up to 4,096 in tables), ≤ 200 steps, children ranked by the network, hash dedup; optional retries at $2W, 4W$. Cost $O(TWA)$ evaluations, $A = 2N$ on the Square.

Results. Optimal on all 3×3×1 and 4×4×1; equals the optimum at $N = 5$; best (with SymPort) up to $N = 8$. Wider beams help: 8×8×1 solved 3/10 at $W = 16$, all from $W = 1024$. Square only — we did not train cubes at CayleyPy's scale (they used ∼8 billion examples for 3×3×3).

Q: Why not run CayleyPy on cubes?

A: Each size needs its own network trained on billions of states; the paper's 3×3×3 model uses 8B examples. That is beyond our compute; we reproduce it where per-size training is cheap (Squares) and say so as a limitation.

Q: Is the walk length a good target? It overestimates distance.

A: Yes it overestimates (random walks wander); it is still monotone enough in expectation to rank children in a beam, which is all beam search needs. That is CayleyPy's own design; we keep it.

4.5 OneNet (new): one network for every size

Idea. Replace per-size sticker networks with a model over *cluster tokens*, so the same weights apply at every N.

- **Tokens.** One token per cluster: one-hot colours of its stickers in the canonical (N-invariant) order, projected by a linear map shared by all clusters of the same type; plus a learned embedding of the cluster kind and a small MLP of geometric features (two in-face depths$/(N-1)$, and $N/20$).
- **Encoder.** Transformer encoder (pre-norm), *no positional encoding* — tokens are a set (Set Transformer idea).
- **Additive head.** Each token predicts a non-negative cost; sum them, add a small global correction. Matches the structure (distance grows with the number of unsolved clusters) and extends to any token count.
- **Training (`m5a_tokens/train.py`).** Fresh non-backtracking walks on every training size $N = 4..8$; target walk length k; loss = MSE of $(\hat{k} - k)/(N^2/\ln N)$ so all sizes count on one scale. AdamW, lr $3 \cdot 10^{-4}$, weight decay 10^{-4}, OneCycle schedule (5% warm-up), gradient clipping 1.0, batch 512.
- **Models.** Main table: GPU model, $d = 192$, 4 layers, 6 heads, FF 512, 20,000 steps of 512 on a Colab T4 (1.56M parameters, `checkpoints/m5a/square.large.pt`). Small CPU model: $d = 64$, 2 layers, 4 heads, 2,500 steps, 198,658 parameters, 35 CPU-minutes.
- **Search $N \le 7$:** full beam search. **$N \ge 8$: staged beam search** outside-in (frame = corner, center, edge-cross clusters; then edge clusters two at a time; then cross and interior clusters three at a time). Each stage has an exact goal: its pieces and all earlier ones show solved stickers, so later stages cannot undo earlier work. Actions: moves touching a target, plus macros $c\,r\,c\,r$ (changes only the cluster at the crossing, Demaine Lemma 2) and conjugates $r\,c\,r$, $c\,r\,c$. Stage heuristic = sum of OneNet's token costs over targets and the earlier clusters the actions can move. If a stage's projection has $\le 10^5$ states, BFS it and solve optimally by lookup (the frame always: it is a 3×3×1). Width 128, retry ×4 up to 2,048.

Results. Solves Square sizes it never saw (up to 14×14×1); second only to per-size networks up to $N = 7$; above $N = 7$ hand-designed schedules (NetGain) are shorter. Distance correlation (Pearson r) on fresh walks: 0.48–0.67 on training sizes, 0.66 / 0.65 / 0.53 at unseen $N = 10$ / 14 / 20. 10×10×1 ≈ 9 s, 14×14×1 ≈ 90 s on T4. $\Theta(N^2)$ moves ($\Theta(N^2)$ stages).

Q: How do you know the network generalises and isn't just the staged search doing the work?

A: Two pieces of evidence: the correlation table at unseen sizes (10, 14, 20) stays near the training-size values; and below $N = 8$ there are no stages — the full-puzzle beam search uses the network alone and is second only to per-size networks. Honest answer: at large N the stage structure carries much of the load.

4.6 NetGain (new): net-gain scheduling of bulk commutators

Idea. Keep Demaine's bulk moves, but apply a commutator type to a block of layers when the *block as a whole pays*, not only when every cell helps.

$$(X^*, Y^*) = \arg\max_{X \cap Y = \emptyset} \sum_{(i,j) \in X \times Y} g_t(i,j) \;-\; \lambda \left(2|X| + 2|Y| + 4 + 2|s_t|\right)$$

$g_t(i,j)$ = change in solved pieces of the cluster cell (i,j) acts on (can be 0 or negative); λ = best pieces/move of any single-cluster 3-cycle now; s_t = the type's setup; every cell must be a verified pure 3-cycle. A block is applied exactly when its pieces/move beat the best single fix.

- Maximise by alternation (rows given columns, columns given rows) from the 3 best rows; 24 types with largest positive gain.

- Exact scoring: apply each candidate to the state; also checks nothing outside the clusters moved.
- Setup sharing: if a type's setup equals the previous step's, charge $2|s_t|$ less ($s_t' s_t$ cancels in canonicalisation).
- One-step lookahead for $N \leq 15$: re-score the 4 best by the best step after them.
- Everything else (wing parity, 3×3×3 stage, Square frame/edges, type bank, single-cluster fallback) is DEMAINE's, so the difference in results is the schedule alone. Square interiors: compute both schedules, keep the shorter.

Results. 20×20×20: 229 of 333 center rounds are bulk; 4,925 HTM (no lookahead, the default at $N = 20$; 4,794 with it in the ablation) vs DEMAINE's 6,612. Random cubes, 20 states, paired 95% CIs: significantly shorter than DEMAINE from $N = 5$ (not at $N = 4$); -17.2% at $N = 10$. 2.6 minutes per 20×20×20. Grows between $N^2/\log N$ and N^2 empirically — no proved bound. Best single solver on large Squares. Cost: round $O(N^3)$, $O(N^2)$ rounds; lookahead $\approx \times 4$ time.

Q: Why is NetGain so weak on Santa?

A: Santa needs the exact goal and QTM, and KMCoders' insertion pipeline is built for that; our cluster methods solve clusters sequentially with long 3-cycle sequences. Its constant is 2.8–3.3× the published lengths. The advantage of NETGAIN shows on random states and in growth rate.

4.7 KMC-Repro (reproduction of KMCoders from the write-up)

The insertion identity. For moves $a_0 \dots a_{T-1}$, suf_t = permutation of $a_t \dots a_{T-1}$, inv_t its inverse. Inserting B before move t changes the final state F to $F[C]$ with $C = \mathrm{inv}_t \circ B \circ \mathrm{suf}_t$. So with all suffixes stored, a candidate at any time is scored by one gather, no replay.

1. Special parts: fixed centers home (middle slices); even N: corners as a 2×2×2, optimal QTM via IDA*-PDB's corner PDB (matches KMCoders' 2×2×2 lengths exactly); odd N: central 3×3×3 by Kociemba (KMCoders used an optimal solver).
2. Wing parity: one inner-slice quarter turn per odd wing cluster.
3. Simulated annealing over the sequence: objective QTM + α·wrong stickers, $\alpha = 2.5$, $6000N^2$ iterations, temperature $2 \to 0.05$; only inner non-middle quarter turns edited; neighbours keep every cluster's permutation even; scored exactly in $O(6N^2)$ by the identity.
4. Greedy insertion of 4-move commutators $[x, y]$ (inner–inner, inner–face, face–inner), best pieces fixed per move net of cancellations, over 256 sampled times.
5. Finish each cluster with QTM-optimal pure 3-cycles at the cheapest time.

Result. 1.61–1.71× the published lengths; still our best non-KMCoders cube method from $N = 6$. **What the write-up leaves out** (seen later in their code): bulk commutators $[D_I, R_J]$ over slice sets chosen by annealing; exact count of the minimum 3-cycles each cluster still needs, inserting only 3-cycles that lower it, beam of 10^5 for small N; optimal central 3×3×3.

Q: Did you look at their code before reproducing?

A: No: KMC-REPRO was built from the written description only, before looking at their code. That is the point of the comparison: the gap between KMC-REPRO (1.6–1.7×) and one run of their program (1.05–1.18×) measures what the write-up leaves out.

4.8 External baselines

- KMC-PUBLISHED: KMCoders' final submission, replayed in our engine; per-puzzle best over the whole competition (many runs, much tuning). All 184 reach the exact goal.
- KMC-ONE RUN: their Rust `solve_cube`, built from source, one run per state; our input matches their files byte for byte (unit test). Odd N: their central 3×3×3 came from a cache made by an external optimal solver; we supply it with Kociemba (the program's own fallback). A two-line patch exposes seed and annealing length via environment variables; defaults unchanged. 1.05–1.18× published.
- DWALTON76: open NxNxN reduction solver with ≈8 GB lookup tables. Wide turns expanded. At $N = 9$ its answers end in another whole-cube orientation; Santa's goal is exact, so we append the shortest whole-cube rotation, counted as N slice turns per quarter turn (9–18 moves).

- Components: `kociemba` package (two-phase); Rokicki's `nxopt` (QTM-optimal 3×3×3, 1.42 GB table, one long-lived process).

Licence. KMCoders' repository has no licence file, so we release only our wrapper and the two-line patch, never their code.

5 SymPort in depth (the main method)

5.1 Idea

A deterministic solver far from optimal is not equally far on every input. The 48 symmetries of the cube (24 rotations × mirror) turn one scramble into 48 *different but equivalent* inputs. Run the solver on k copies, map each answer back, keep the shortest. Square: 4 rotations × mirror = 8 copies. Seed-free source of diversity.

5.2 The proposition (be able to prove this on the board)

Let R be the sticker permutation of a rotation or mirror image, σ the colour relabelling with $\sigma(s^*[R]) = s^*$. Copy of s: $s' = \sigma(s[R])$.

Claim. For every move q there is a move $\phi_R(q)$ of the same cost with $R \circ q \circ R^{-1} = \phi_R(q)$. Hence if $q_1 \dots q_L$ solves s' exactly, then $\phi_R(q_1) \dots \phi_R(q_L)$ solves s exactly, with the same QTM, HTM and STM.

Proof. A rotation or reflection maps every slab onto a slab, so conjugating a slab turn by R is a turn of the image slab by the same angle (sense reversed for a reflection): the move set is closed under ϕ_R and costs are preserved. Then $s'[q_1 \cdots q_L] = \sigma(s[R][q_1 \cdots q_L])$; substitute $q_i = R^{-1}\phi_R(q_i)R$, the inner $R^{-1}R$ pairs telescope, giving $\sigma(s[\phi_R(q_1) \cdots \phi_R(q_L)][R])$. This equals $s^* = \sigma(s^*[R])$ iff $s[\phi_R(q_1) \cdots] = s^*$ (R is a bijection). The harness independently re-verifies every mapped-back solution anyway. □

Square: the symmetries permute the 4 solved orientations, so "any solved orientation" is also preserved.

5.3 Configuration

- **Cube inner solver**: KMCoders' program with (i) their beam variant `solve_cube_beam` (write-up recommends it for small N); (ii) 1M instead of 20M annealing iterations in their stage 2 (stage 2 is ≈3/4 of a 4×4×4 run; lengths did not change measurably at 0.5M, 2M or 20M); (iii) odd N: optimal QTM central 3×3×3 from `nxopt` via their cache file (Kociemba's answer defines the position; nxopt solves it optimally). Santa puzzle 210: 34 → 24 QTM for that stage.
- **Copy order**: identity, its mirror, then each other rotation followed by its mirror, truncated to k.
- k **per size** (budget): $N = 4$: 48 beam copies + 8 copies of the default program (20M); $N = 5$–7: 24; $N = 8$–9: 12.
- **Square inner solvers**: every copy solved by each: CayleyPy ($W = 4096$) and NetGain for $N \leq 8$; NetGain and Demaine above.
- Cost: k× inner solver. 17–28 CPU-min per 4×4×4 (48 copies), 28–46 per 5×5×5 (24); 31 and 47 CPU-min per copy at $N = 8, 9$. Memory: inner solver + nxopt's 1.42 GB table (shared process pool).

5.4 Results

- Santa: best at $N = 4$–7 (published second), second at $N = 8, 9$ (published first). Always ahead of one run of their program.
- Per size W/T/L vs published: 4: 24/9/10; 5: 11/3/5; 6: 4/2/0; 7: 2/0/2; 8: 1/0/2; 9: 1/0/2.
- Square: best at all 18 sizes; reaches the optimum where known ($N \leq 5$); improves on NetGain above $N = 8$ by up to ≈7%.
- Odd N: the optimal central 3×3×3 matters as much as the portfolio: with Kociemba's stage the same 24-copy portfolio averaged 102.0 on the first three 5×5×5 puzzles, vs 95.3 with nxopt and 100.7 published.
- 4×4×4 figure: 48 copies spread over 18–28 QTM per puzzle (copies 60–89, mean 75.2); the published solution sits in the lower tail; best-of-k still falling at $k = 48$.

5.5 Why it stops winning at $N \geq 8$ (the analysis you must own)

Split the result into **gap** = (median copy − published)/published and **gain** = (median copy − best copy)/median. SymPort wins exactly where gain > gap.

N	k	CPU-min/copy	SD (QTM)	rel. spread	gain	gap	SymPort	published
4	48	0.3	4.5	6.0%	14.2%	12.8%	64.6†	66.7
5	24	1.0	5.0	4.8%	9.4%	8.0%	95.0	97.1
6	24	6.3	6.5	3.8%	7.0%	5.8%	158.3	161.0
7	18	10	7.0	3.2%	5.8%	4.4%	205.2	208.8
8	12	31	7.4	2.4%	3.3%	4.6%	294.7	291.3
9	12	47	7.3	2.0%	3.3%	5.4%	361.3	354.7

† beam copies only; Table 2's 64.1 adds the 8 default-program copies. k = median copies finished (a few copies at $N = 5$, 7 were lost to the memory limit).

Three causes:

1. **The spread does not grow.** SD of copy lengths stays ≈4.5–7.4 QTM while lengths grow as N^2; relative spread falls 6% → 2%. Symmetric copies differ in early stages and tie-breaks; the long late stages come out much the same.
2. **Fewer copies.** Copies cost 31–47 CPU-min at $N = 8$–9, so 12 instead of 24–48.
3. **Published is itself a portfolio**: best of many runs and settings over the competition.

Projection. A normal model fitted per puzzle reproduces the measured best-of-12 within half a quarter turn at $N = 8$, 9, and projects best-of-48: 290.2 vs 291.3 at $N = 8$ (just wins), 357.3 vs 354.7 at $N = 9$ (still loses), at ≈25 and 38 CPU-hours per puzzle. So symmetry alone runs out; it needs more diverse copies or a stronger base solver.

Expected min of k normals: $\mathbb{E}[\min] = \mu - e_k\sigma$, $e_{12} \approx 1.63$, $e_{48} \approx 2.25$ (computed by numerical integration in `santa_part.py`).

5.6 Why we stopped at $N = 9$

After two consecutive sizes where SymPort loses to KMC-published, with the gap widening from $N = 8$ (3.4 QTM) to $N = 9$ (6.6 QTM), and the projection saying 48 copies would not close $N = 9$, we take $N \leq 7$ as its winning range and leave larger cubes to future work. *Phrase it as "best method up to $N = 7$", never "optimal" — every method is ≈ 2× the lower bound.*

5.7 The memory incident (be ready; be honest)

In the first $N = 8/9$ runs, three copies ran in parallel, each KMCoders search taking 3–4.7 GB, plus up to three idle nxopt servers (1.4 GB each, a pool kept for reuse) on a 15 GB machine: the OS killed searches, and killed copies were recorded as failures. At $N = 9$ only 1–2 of 12 copies had finished. We fixed it (2 copies at a time from $N = 8$; idle nxopt processes freed between sizes; failed copies no longer cached, and their errors recorded) and **reran only the failed copies**. That is not cherry-picking: the failures were caused by the machine, not by the method, and every finished copy was kept. Effect: $N = 9$ mean 374.0 → 361.3; $N = 8$ unchanged (the new copies were all longer than the best already found).

5.8 Seed control (not run)

The obvious objection: "48 random seeds of KMCoders' program might give the same diversity." We have not run the equal-compute seed comparison. It is the first experiment in future work and is listed in limitations. Argument for symmetries: they are seed-free and need no change to the program; the program's randomness enters mainly through annealing, while symmetry changes the whole input. But we cannot claim symmetries beat seeds.

6 The counting lower bound

Every state at distance d is the end of at least one *canonical* word of length d. If $c(d)$ counts canonical words of cost d, the radius L satisfies $\sum_{d \le L} c(d) \ge |G|$.

- Cube, QTM: a canonical word is a sequence of blocks; a block turns a non-empty set of layers of one axis, each by $+1$, -1 or 2 quarter turns (cost 1, 1, 2), consecutive blocks on different axes. Block GF $f(x) = (1+x)^{2N} - 1$ (each of N layers: nothing, or x twice, or x^2: $1 + 2x + x^2 = (1+x)^2$); words $G(x) = 1 + 3f/(1-2f)$ (3 axes to start, 2 choices after). Growth $b = 1/x^*$ with $2f(x^*) = 1$:

$$b = \left(\left(\tfrac{3}{2}\right)^{1/(2N)} - 1\right)^{-1}, \qquad L \ge \frac{\ln |G|}{\ln b} - O(1).$$

- $|G|$ counted cluster by cluster: $8!\,3^7$ corners, $12!\,2^{11}$ middle edges, $24!$ per wing cluster, $24!/(4!)^6$ per non-fixed center cluster (same-colour centers indistinguishable). Parity couplings (≈ 1 bit per cluster) ignored.
- Square: rows commute with rows, columns with columns; words alternate non-empty row sets and column sets: $f = (1+x)^N - 1$, $b = 1/(2^{1/N} - 1)$.
- Valid because words are over-counted (several words reach one state). Exact word count agrees with the formula within one move (unit test `test_counting_bound.py`).

Why $N^2/\log N$: $\ln |G| = \Theta(N^2)$ (number of clusters), $\ln b \approx \ln(2N/\ln 1.5) = \Theta(\log N)$.

Calibration. 3×3×1: 768 states, bound 4, God's 6 (1.50); 4×4×1: 82,944, 6, 10 (1.67); 5×5×1: 10,616,832, 8, 12 (1.50). 4×4×1 and 5×5×1 are the first exact values we know of (colour goal, any of 4 orientations; BFS). The bound uses $|G|/4$ (four solved orientations).

What it says. Published Santa lengths are 1.81× the bound at $N = 4$, 1.99× at 7, 2.22× at 10. Lengths grow like N^2 ($\approx$4.2–4.6N^2), the bound like $N^2/\log N$: the final stage of KMCoders' method fixes $O(1)$ pieces per 3-cycle. Keeping Demaine-style parallelism into the endgame (what NetGain does) is the way to close it.

Limits. It is a lower bound on the radius used as an estimate of typical distance; it cannot certify any single solution; the Square says the true worst case is 1.5–1.7× the bound.

7 Experiments and statistics

- **Hardware.** All CPU except GPU training (Colab T4). OneNet and CayleyPy Square evaluations on a 2-core machine or the T4; everything else on a shared 4-core, 15 GB machine (wall times indicative).
- **Table 1 (Squares, HTM)**: 7 systems, $N = 3$–20, red = best, green = second; only methods that solved every instance are ranked. SymPort best at all 18 sizes.
- **Table 2 (Santa, QTM)**: 8 systems, $N = 4$–9.
- **Paired CIs.** 20 random cubes per size $N = 3$–10, paired differences NetGain − Demaine on the same states, 95% intervals: significant from $N = 5$; −1.5% (4, n.s.), −3.7% (5), −7.7% (6), −9.6% (7), −11.8% (8), −16.3% (9), −17.2% (10).
- **Reproductions vs originals.** PDB sizes = Korf's; 3×3×1 diameter matches the published value; CayleyPy behaves as reported (wider beam helps, optimal at small N); KMC program one run 1.05–1.18× published; KMC-Repro 1.61–1.71×.
- **Figure 5**: length/$(N^2/\ln N)$ for (a) random Squares with every Square system, (b) random cubes (only the three cluster methods ran there), (c) Santa cubes with every Table 2 system. A flat line = worst-case rate; every curve still rises.
- **Figure 6**: Santa length / counting bound.
- Ablations (`paper/results/ablations`): NetGain lookahead (4,794 vs 4,925 at 20×20×20), setup sharing; earlier staged solver components.

8 Code map and how to run things

Path	What
`rubiks/engine/`	`puzzle.py` (geometry, moves, apply), `moves.py` (canonical form, metrics), `orbits.py` (clusters, signatures), `projection.py`, `checker.py`, `random_states.py`
`rubiks/search/`	`beam.py` (shared beam search), `macros.py` (3-cycle/commutator generation)
`rubiks/solvers/base.py`	solver interface, `Budget`, registry (`@register`, `get_solver`)
`rubiks/solvers/m1...m7_*`	the methods (see names table)
`.../symmetry.py`	the 48 symmetries, `rotated`, `map_back` (SymPort's proposition in code)
`rubiks/solvers/` (shared)	`cluster_tools.py`, `three_stage.py`, `square_plan.py`: shared stages for Reduction/Demaine/NetGain
`rubiks/solvers/external/`	`kmcoders_run.py` (wrapper, Santa input format, stage-1 cache), `nxopt.py` (server pool), `dwalton76.py`, `santa_kmcoders.py` (published solutions)
`rubiks/bench/`	`harness.py`, `santa2023.py` (adapter), `datasets.py`, CLI `python -m rubiks.bench`
`rubiks/analysis/`	`counting_bound.py`: the bound
`experiments/`	`m7_santa.py` (SymPort campaign; copy cache in `M7_COPY_CACHE`), `m7_square.py`, `kmc_one_run.py`, `b3_santa.py`, `m7_seed_control.py` (not run), `run_batches.py`, `e1`–`e8` scripts
`docs/results/`	every JSONL result file (gitignored, force-added)
`paper/scripts/`	`make_tables.py` regenerates every number, table and figure (with `santa_part.py`, `main_tables.py`)
`checkpoints/`	PDBs, CayleyPy and OneNet models
`tests/`	17 test files: engine, checker, beam, bound, harness, Santa adapter, KMCoders input, each method

Typical commands:

- `pytest -q`
- `python -m rubiks.bench make --puzzle cube --N 10 --seed 0`
- `python -m rubiks.bench run --solver m5c_schedule --puzzle cube --N 10 --limit 2`
- `python paper/scripts/make_tables.py`, then `pdflatex`/`bibtex` in `paper/`
- External programs need `KMCODERS_DIR`, `NXOPT`, `NXOPT_DIR`, `DWALTON76_DIR`.

Reproducibility claims you can make: every instance regenerated from a seed; every solution re-verified by the harness; every number in the paper generated by script from result files; git commit stored in each result row.

9 Question bank

9.1 Framing and novelty

Q: What exactly is new here?

A: Three methods (SymPort, NetGain, OneNet), the calibrated counting bound with the first exact God's numbers for 4×4×1, 5×5×1 that we know of, and one open engine/harness where all eight methods and three baselines run on the same instances. Five methods are reproductions, clearly labelled.

Q: Isn't SymPort just running someone else's solver many times?

A: The inner solver is theirs, and we say so. The contribution is (i) the observation that symmetric copies are a seed-free source of diversity with a proof that lengths are preserved exactly, (ii) the configuration (beam variant, short annealing, optimal central 3×3×3), and (iii) the measured result and its limits, including the analysis of why it stops working.

Q: Why the Rubik's Square?

A: It shares the cluster structure of the cube but small sizes are exactly solvable, so it is where we can compute optima, calibrate the bound and train learned models cheaply.

9.2 SymPort

Q: Prove that mapped-back solutions are valid and the same length.

A: See the proposition (Section 5.2). Key step: conjugating a slab turn by a symmetry gives a slab turn of the same cost; the $R^{-1}R$ pairs telescope.

Q: Mirror images: is a reflection a legal cube symmetry?

A: Not physically, but as a sticker permutation it maps the move set onto itself (quarter turns to opposite quarter turns of the mirrored slice), so the proof goes through. We relabel colours with σ so the copy has a valid colour scheme.

Q: Why 12 copies at $N = 8$–9, not 48?

A: Budget: 31–47 CPU-min per copy; 48 copies would be 25–38 CPU-hours per puzzle. The model says 48 would only just win at 8 and not at 9.

Q: Is the normal model justified?

A: It reproduces the measured best-of-12 within half a quarter turn at $N = 8$, 9; we use it only for a projection and say so. Copy distributions are roughly bell-shaped with 12–48 samples; the claim is directional.

Q: Your wins at $N = 6$–9 are on 3–4 puzzles. Significant?

A: No; we say so. Per-size margins at small n are not significant differences. The pooled result over $N = 4$–7 is 72 puzzles, 41 wins, 14 ties, 17 losses.

Q: Why did you stop at 9? Isn't that convenient?

A: It is the opposite of convenient — we report two losses. We stopped because two consecutive losses with a widening gap, plus the projection, say the method in this configuration has reached its limit; each further puzzle costs many CPU-hours.

Q: You reran failed copies — isn't that selective?

A: Only copies the OS killed for lack of memory were rerun; every copy that finished was kept, good or bad. The rerun made $N = 8$ no better and $N = 9$ better but still a loss.

9.3 Learned methods

Q: Training data and target?

A: Fresh non-backtracking random walks from solved; target = step count k; MSE (OneNet: normalised by $N^2/\ln N$). Fresh data every batch/epoch.

Q: Why a set transformer without positions?

A: Clusters form a set; their identity comes from the type embedding and geometric features, and dropping positional order lets the token count change with N.

Q: Why an additive head?

A: Total distance grows with unsolved clusters; a sum of per-cluster costs extends to any number of tokens and gives per-cluster costs that the staged search can use for subsets.

Q: Overfitting / leakage?

A: Training states are random walks; test states are independent seeded long random walks; the Square optimum tables are separate. No test state is used in training.

Q: Why are learned methods not on cubes?

A: Compute (billions of examples per size). Stated limitation.

9.4 Search and engineering

Q: How is the beam deduplicated?

A: Random 64-bit linear hash of the packed state, also across the last three steps.

Q: Why canonical order in IDA* and in the bound?

A: It removes redundant orderings of commuting moves and immediate repeats; the same word model makes the bound tight-ish and the search smaller.

Q: How do you know the external programs' answers are right?

A: The harness parses them into engine moves and re-applies them; a solution counts only if our engine reaches the goal.

9.5 Bound and theory

Q: Derive b.

A: See Section 6. Each layer contributes $(1+x)^2$; minus the empty block; words alternate axes with 3 then 2 choices; the dominant pole solves $2f(x) = 1$.

Q: Is it a bound on God's number or on typical distance?

A: On the radius covering $|G|$ states, so on God's number; we use it as an estimate of typical optimal distance and say so.

10 Weak spots and inconsistencies to be ready for

- **21% vs 17% at $N = 10$.** The abstract's "21% shorter than DEMAINE at $N = 10$" (`\FiveCGainTen`) comes from the 2-state main random-cube table (1,196 vs 1,509 HTM). The 20-state paired CI table gives -17.2% at $N = 10$, and the slides show 17.2. Both are real; say which table each comes from. Consider changing the abstract to the 20-state number (17%), which is the statistically stronger one.
- **OneNet correlation on the slides.** Slide 13 says "$r \geq 0.70$ at unseen $N = 10, 14, 20$"; the paper's table gives 0.66, 0.65, 0.53. Fix the slide or be ready to correct it.
- **KMC one run ratio.** Slides say 1.07–1.18× published; the data give 1.05 at $N = 10$ (1.05–1.18).
- **$N = 10$.** Table 2 stops at 9, but Figure 6 and the bound ratios still use $N = 10$ for other systems; SYMPORT was not run at 10 (only 4 copies of one puzzle: best 462 vs 454 published). The text says we stopped.
- **Seed control not run** (equal-compute random seeds). Biggest methodological gap; listed in limitations and future work.
- **Few puzzles at $N \geq 6$** (3–4 per size); wins there are not significant.
- **Checker is necessary-only**; random states are long walks, not an exact uniform sampler.
- **KMC-one run uses Kociemba for odd-N central 3×3×3** (their program's fallback), whereas the published solutions used an optimal solver; SYMPORT uses nxopt (optimal). So part of SYMPORT's gain over one run at odd N is the optimal stage, which we measured separately (5×5×5: 102.0 Kociemba vs 95.3 nxopt).
- **Copies lost to memory** at $N = 5$ and 7 in the original runs (median k 24 and 18). They still won; a rerun could only lower those numbers.
- `m5b_stages` exists in the code but not the paper (dropped variant).
- **Licence**: we do not redistribute KMCoders' code.

11 Future work (as in the paper)

1. More diverse copies: seeds, tie-breaks, beam widths inside KMCoders' later stages; first, equal-compute symmetries vs seeds.
2. Recombination instead of selection: map copies to one frame and keep the shortest *stage* from each.
3. Adaptive budget (successive halving): cheap early stages on all 48 copies, finish only the most promising.
4. A stronger base solver: wider beam, or NETGAIN's bulk commutators in the endgame (also attacks the $\log N$ gap).
5. Remaining Santa sizes ($N = 10$, 19, 33); cube networks at CayleyPy's scale; an exact uniform sampler.